

软件设计模式 练习题集

(共 10 题: 5 道画图题 + 5 道填空题)

(文档内画图和代码需要一定学科基础, 建议复习一些相关知识再一点点看)

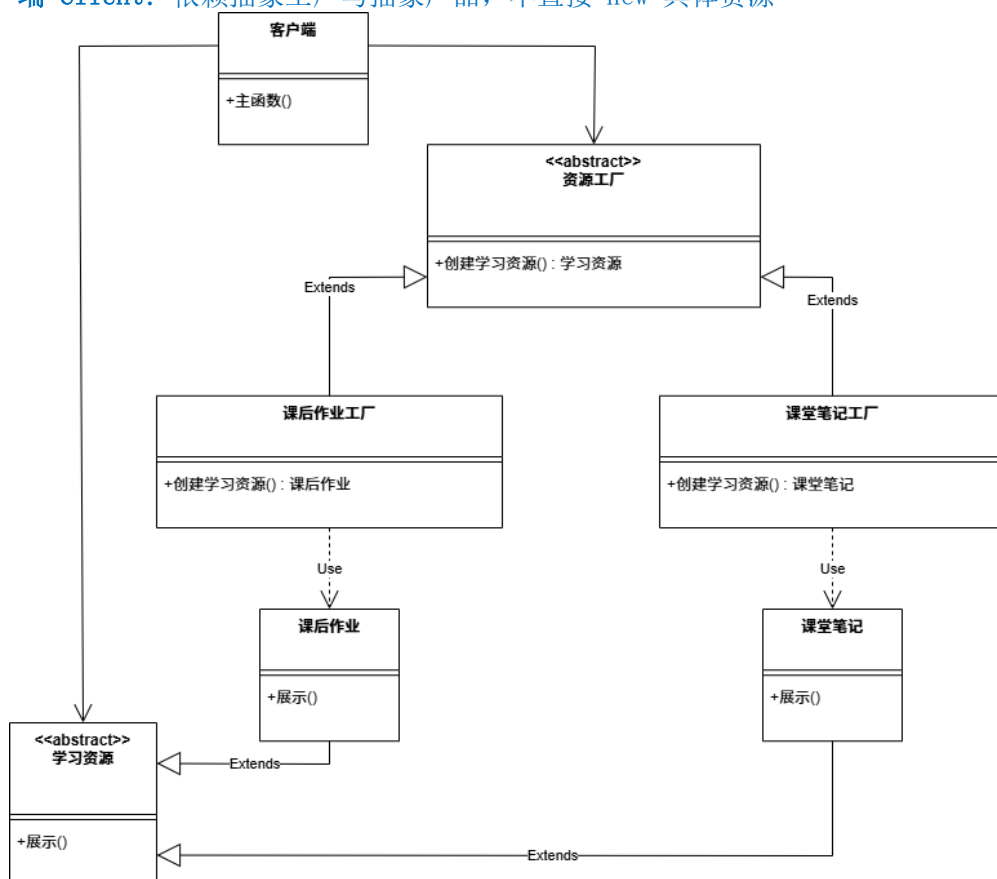
一、画图题 (UML 类图绘制)

题目 1 (工厂方法模式)

在线学习平台需要实现不同类型学习资源的创建与展示, 涉及课后作业、课堂笔记两种在线学习资源, 要求使用工厂方法模式设计该资源管理系统, 请绘制 UML 类图。

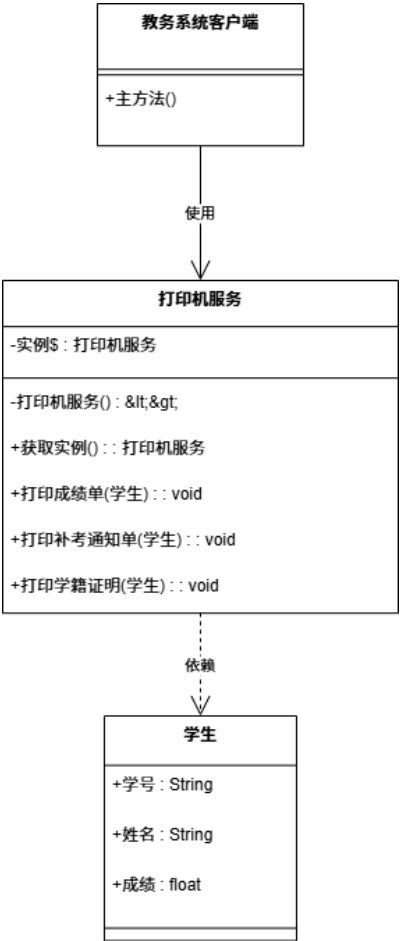
工厂方法模式核心角色

1. **抽象产品**: 定义所有学习资源统一规范接口
2. **具体产品**: 课后作业 Homework、课堂笔记 ClassNote, 实现抽象产品接口
3. **抽象工厂**: 定义创建产品的抽象方法 `createResource()`
4. **具体工厂**: HomeworkFactory、ClassNoteFactory, 各自实现创建对应产品的方法
5. **客户端 Client**: 依赖抽象工厂与抽象产品, 不直接 new 具体资源



题目 2 (单例模式)

学校教务管理系统需要实现学生成绩单、补考通知单、学籍证明在线打印功能。系统如果创建多个打印服务对象，会出现打印队列错乱、重复提交打印任务、打印机端口占用异常问题，因此规定整个程序运行期间，创建打印机服务对象。要求使用单例模式来设计这个打印服务类，同时提供打印方法，请绘制 UML 类图。



单例模式设计解析

1. 核心问题与模式意图

在教务系统中，若创建多个打印服务对象，将导致打印队列错乱、任务重复提交、打印机端口占用异常。单例模式通过**全局唯一实例**强制集中管理所有打印请求，从根本上避免多实例竞争资源。

2. 类图中角色分解

类名	职责
打印机服务	单例类，封装打印机的唯一访问入口和所有打印功能。
学生	承载待打印数据（学号、姓名、成绩），是打印方法的参数类型。
教务系统客户端	调用 打印机服务. 获取实例() 获得唯一对象，再调用具体打印方法。

3. 关系线解析

关联（实线开口箭头）：教务系统客户端 --> 打印机服务

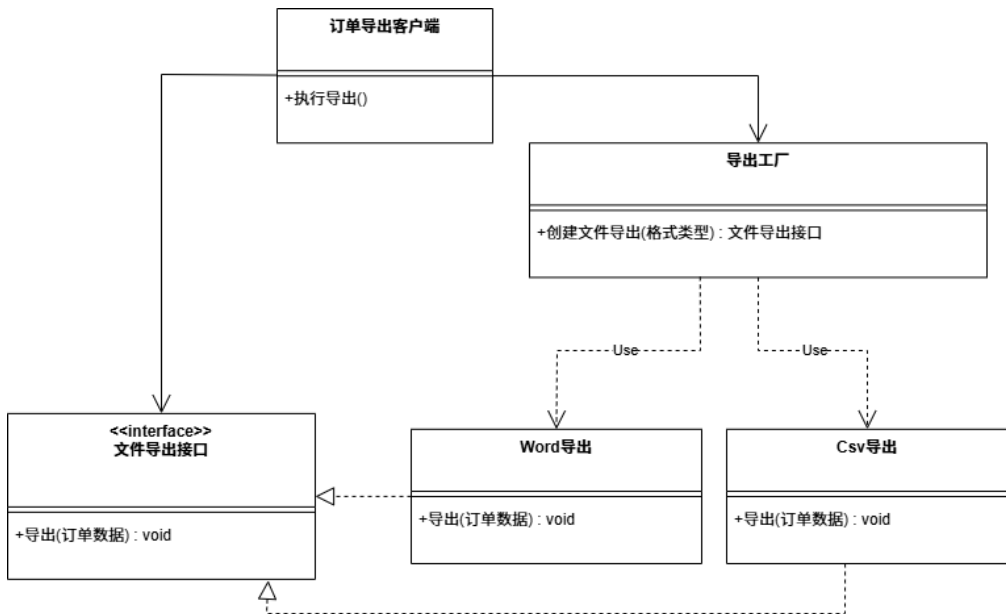
客户端持有 打印机服务 的引用（通过调用 获取实例() 获得），并持续使用其方法，这是一种结构性的“拥有/知道”关系。

依赖（虚线开口箭头）：打印机服务 ...> 学生
打印机服务 的各个打印方法以 学生 对象作为参数，该依赖是临时的、局部的，只在方法调用期间存在。

泛化（本图未出现）
单例模式不涉及继承，因此类图中没有空心三角箭头。

题目 3（简单工厂模式）

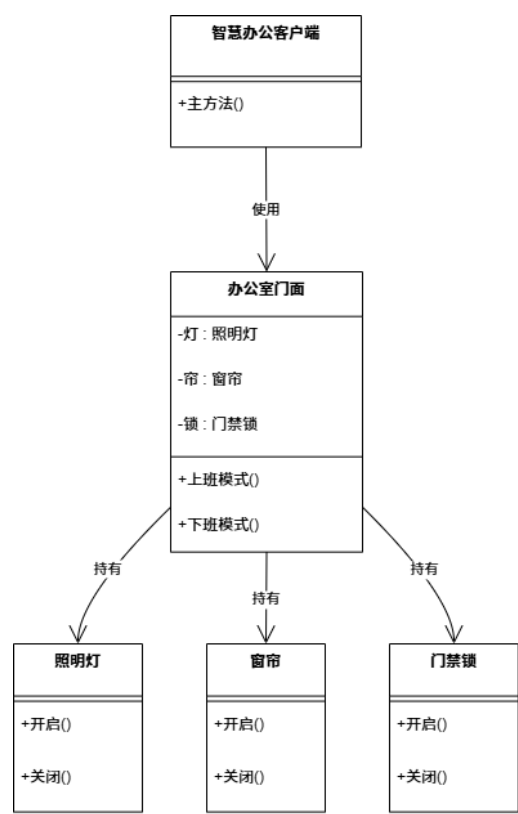
某电商平台后台需要实现订单数据导出功能，支持 Word 订单文档、CSV 表格文件两种格式导出。请采用简单工厂模式，设计一个工厂类（ExportFactory），该工厂提供了 createFileExport() 方法，该方法会根据不同参数创建对应文档导出类实例（如 WordExport、CsvExport）。文档导出类都实现了文件导出接口（FileExport）。请绘制该类图。



类/接口	图中名称	职责
抽象产品	文件导出接口	定义所有导出类的统一行为接口，声明 导出(订单数据) 方法。客户端只依赖该接口，无需关心具体实现。
具体产品	Word 导出、Csv 导出	分别实现 导出(订单数据)，完成将订单数据写入 Word 文档或 CSV 文件的具体逻辑。
工厂类	导出工厂	提供静态方法 创建文件导出(格式类型)，根据传入的参数（如“WORD”、“CSV”）判断并实例化对应的具体产品，然后以 文件导出接口 类型返回。
客户端	订单导出客户端	调用工厂获取导出对象，再调用 导出() 方法，全程只与抽象接口交互。

题目 4（外观模式）

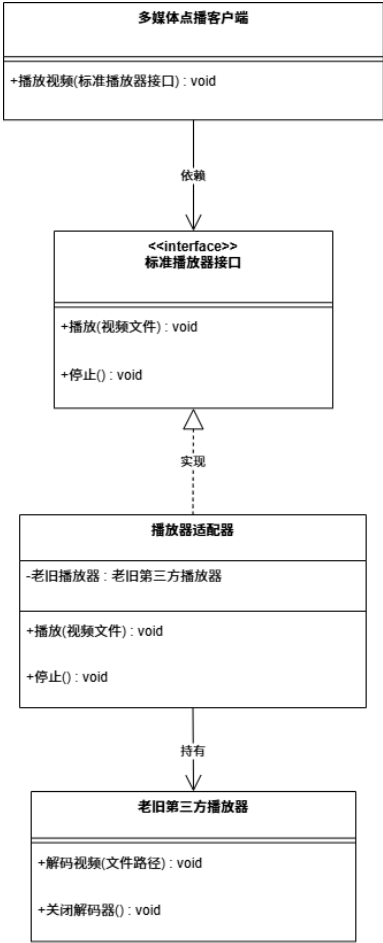
现开发一个企业智慧办公系统，需要通过统一门面类 OfficeFacade 提供两种场景模式：上班模式 workOnMode()、下班模式 workOffMode()。系统内部包含多个相互独立的子系统设备：照明灯(Light)、窗帘(Curtain)、门禁锁(DoorLock)。每一个设备子系统都各自拥有 turnOn() 开启、turnOff() 关闭方法，客户端不需要逐个操控单个设备，只调用门面类对应的模式方法即可一键批量联动所有设备，请使用外观模式完成整体设计。请绘制 UML 类图。



角色	图中类名	职责
子系统	照明灯、窗帘、门禁锁	各自独立完成特定的设备操作，提供 <code>开启()</code> 和 <code>关闭()</code> 方法。它们不知道门面类的存在，可独立演化。
门面	办公室门面	持有所有子系统的引用，组合出高层场景方法。 - <code>上班模式()</code>：内部依次调用 <code>灯.开启()</code>、<code>帘.开启()</code>、<code>锁.开启()</code>（模拟窗帘拉开、门禁解锁等）。 - <code>下班模式()</code>：依次调用 <code>灯.关闭()</code>、<code>帘.关闭()</code>、<code>锁.关闭()</code>（模拟关灯、拉帘、上锁）。
客户端	智慧办公客户端	只需调用门面的 <code>上班模式()</code> 或 <code>下班模式()</code> ，一键完成所有设备联动，完全解耦子系统。

题目 5（对象适配器模式）

校园多媒体点播系统定义了统一视频播放器标准接口，规定系统内播放器必须遵循该接口规范完成播放操作。项目前期接入一款老旧第三方视频解码播放器，该第三方源码无法修改，其内部方法命名、调用方式和现有系统标准接口不一致。为在不修改原有第三方播放器代码、不改动现有系统业务代码的前提下，让老旧播放器适配接入点播系统，要求使用对象适配器模式完成接口兼容改造。请简述改造思路，并画出类图（可用 mermaid 代码表示）。



角色	图中类名	职责
目标接口	标准播放器接口	系统定义的统一视频播放规范，声明 <code>播放()</code> 和 <code>停止()</code> 方法。客户端只依赖此接口。
适配者	老旧第三方播放器	已存在的老旧视频解码器，无法修改源码。它提供了 <code>解码视频(文件路径)</code> 和 <code>关闭解码器()</code> 方法，但命名和参数与目标接口不一致。
适配器	播放器适配器	实现 <code>标准播放器接口</code> ，内部持有 <code>老旧第三方播放器</code> 对象的引用。在 <code>播放()</code> 方法中调用 <code>老旧播放器.解码视频()</code> ，在 <code>停止()</code> 方法中调用 <code>老旧播放器.关闭解码器()</code> ，完成接口转换。

客户端	多媒体点播客户端	系统的业务类，通过 标准播放器接口 使用播放功能，完全不知道适配器内部包裹了老旧播放器。
-----	----------	--

二、填空题（代码补充）

题目 1（工厂方法模式）

请补充代码：

```
// 抽象产品
public interface LearningResource {
    // 展示学习资源的方法
    void show();
}

// 具体产品
public class Homework implements ① _____ {
    @Override
    public void show() {
        System.out.println("工厂方法模式作业");
    }
}

// 具体产品
public class ClassNote implements LearningResource {
    @Override
    public void show() {
        System.out.println("工厂方法模式核心要点、类关系及代码实现");
    }
}

// 抽象工厂
public interface ResourceFactory {
    // 创建学习资源对象
    LearningResource createResource();
}

// 课后作业工厂
public class HomeworkFactory implements ResourceFactory {
    @Override
    public LearningResource createResource() {
```

```

        // 创建并返回 Homework 对象
        return ② _____;
    }
}

// 课堂笔记工厂
public class ClassNoteFactory implements ResourceFactory {
    @Override
    public LearningResource createResource() {
        return new ClassNote();
    }
}

public class Client {
    public static void main(String[] args) {
        // 使用作业工厂创建课后作业
        ResourceFactory homeworkFactory = new HomeworkFactory();
        // 调用工厂对象 createResource 方法，创建课后作业资源对象
        LearningResource homework = ③ _____;
        homework.show();
    }
}

```

第 1 空:

第 2 空:

第 3 空:

题目 1 工厂方法模式

填空答案

① LearningResource

② new Homework()

③ homeworkFactory.createResource()

逐空解析

- ①: Homework 是具体产品，必须实现抽象产品接口 LearningResource，实现其 show() 方法。
- ②: HomeworkFactory 是课后作业工厂，工厂方法需要创建并返回对应具体产品 Homework 实例。
- ③: 客户端通过工厂对象调用工厂方法 createResource()，获取创建好的学习资源对象。

模式核心小结

工厂方法模式：抽象产品、具体产品、抽象工厂、具体工厂一一对应，每个产品专属一个工厂，创建逻辑下沉到具体工厂。

题目 2（单例模式）

请补充代码：

```

class PrintService {
    //创建单例类的实例

```

```

private static PrintService instance = ①_____ ;
//构造函数私有化
private ②_____ () {}
public static PrintService getInstance() {
    //返回单例类实例
    return ③_____ ;
}
public void print(String docName) {
    System.out.println("打印: " + docName);
}
}

public class Test {
    public static void main(String[] args) {
        //获取单例类实例
        PrintService printService = ④_____ ;
        //调用 printService 对象的 print 方法执行打印任务，参数可以任意字符串
        ⑤_____ ;
    }
}

```

第 1 空:

第 2 空:

第 3 空:

第 4 空:

第 5 空:

题目 2 单例模式（饿汉式单例）

填空答案

① new PrintService()

② PrintService

③ instance

④ PrintService.getInstance()

⑤ printService.print("学生成绩单")（字符串参数可自定义）

逐空解析

1. ① 饿汉式：类加载时直接初始化唯一静态实例，全局仅一份打印服务对象。
2. ② 私有化构造方法，修饰符 private，禁止外部通过 new 创建打印服务实例，保证单例。
3. ③ 静态方法 getInstance() 对外提供全局访问点，返回唯一静态成员变量 instance。
4. ④ 客户端不能 new 对象，只能调用类静态方法获取单例对象。
5. ⑤ 拿到单例后调用打印方法，传入任意文档名称作为参数。

模式核心小结

单例模式保证一个类在程序中仅有一个实例，私有化构造 + 静态全局访问点，解决多实例打印队列错乱问题。

题目 3（简单工厂模式）

请补充代码：

```
// 抽象产品
```



```

public interface FileExport {
    void exportOrder(String orderNo);
}

// 具体产品
public class WordExport implements ① _____ {
    @Override
    public void exportOrder(String orderNo) {
        System.out.println("正在导出 【Word 订单文档】");
        System.out.println("订单编号: " + orderNo);
        System.out.println("Word 文件生成完成\n");
    }
}

// 具体产品
public class CsvExport implements FileExport {
    @Override
    public void exportOrder(String orderNo) {
        System.out.println("正在导出 【CSV 表格订单文件】");
        System.out.println("订单编号: " + orderNo);
        System.out.println("CSV 文件生成完成\n");
    }
}

// 工厂类
public class ExportFactory {
    public FileExport createFileExport(String type) {
        if ("word".equalsIgnoreCase(type)) {
            // 创建并返回 WordExport 实例
            return ② _____;
        } else if ("csv".equalsIgnoreCase(type)) {
            return new CsvExport();
        } else {
            throw new RuntimeException("不支持该导出格式: " + type);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        ExportFactory factory = new ExportFactory();
        FileExport wordExport = factory.createFileExport("word");
    }
}

```

```
// 导出 Word 订单，订单号可以是任意字符串
```

```
③ _____;
```

```
}
```

```
}
```

第 1 空:

第 2 空:

第 3 空:

题目 3 简单工厂模式

填空答案

① FileExport

② new WordExport()

③ wordExport.exportOrder("ORD001") (订单号字符串可自定义)

逐空解析

① WordExport 是具体导出产品，实现抽象产品接口 FileExport，重写导出方法。

② 工厂判断 type 为 word 时，创建 Word 导出对象并返回。

③ 获取导出对象后，调用接口定义的导出方法，传入订单编号完成导出。

模式核心小结

简单工厂：一个统一工厂类，根据传入参数判断创建不同产品，所有产品实现同一抽象接口。

题目 4（外观模式）

请补充代码：

```
//照明灯
```

```
public class Light {
```

```
    public void turnOn() {
```

```
        System.out.println("客厅主灯已打开");
```

```
    }
```

```
    public void turnOff() {
```

```
        System.out.println("客厅主灯已关闭");
```

```
    }
```

```
}
```

```
//窗帘
```

```
public class Curtain{
```

```
    public void turnOn() {
```

```
        System.out.println("客厅窗帘已打开");
```

```
    }
```

```
    public void turnOff() {
```

```
        System.out.println("客厅窗帘已关闭");
```

```
    }
```

```
}
```

```
//门禁锁
```

```
public class DoorLock {
```

```
    public void turnOn() {
```

```
        System.out.println("门禁锁已开启");
```

```
    }
```

```

    public void turnOff() {
        System.out.println("门禁锁已关闭");
    }
}

//外观类
public class OfficeFacade {
    private Light light = ①_____ ;
    private Curtain curtain = ②_____ ;
    private DoorLock doorLock = ③_____ ;
    public void workOnMode() {
        System.out.println("====触发【上班模式】====");
        light.turnOn();
        curtain.turnOn();
        doorLock.turnOff();
    }
    public void workOffMode() {
        System.out.println("====触发【下班模式】====");
        light.turnOff();
        curtain.turnOff();
        doorLock.turnOn();
    }
}

public class Client {
    public static void main(String[] args) {
        //创建外观对象
        OfficeFacade officeFacade = ④_____ ;
        //通过外观类对象，触发上班模式
        ⑤_____ ;
    }
}

```

第 1 空:

第 2 空:

第 3 空:

第 4 空:

第 5 空:

题目 4 外观模式

填空答案

① new Light()

② new Curtain()

③ new DoorLock()

④ new OfficeFacade()

⑤ officeFacade.workOnMode()

逐空解析

- ①②③：外观类 OfficeFacade 内部持有所有子系统设备实例（灯、窗帘、门禁），封装所有底层设备。
④ 客户端只需要创建门面对象，无需操作底层子系统。
⑤ 客户端仅调用门面封装好的一键模式方法，由门面内部依次调用各设备开关，简化调用。

模式核心小结

外观模式：提供统一门面，屏蔽多个复杂子系统，客户端仅和门面交互，实现一键批量操作。

题目 5（对象适配器模式）

请补充代码：

```
//目标接口
public interface MediaPlayer {
    void playVideo();
}

//被适配类
public class OldPlayer {
    public void play() {
        System.out.println("老旧播放器：正在解码视频并播放世界杯...");
    }
}

//适配器类，实现目标接口
public class VideoAdapter implements ①_____ {
    private OldPlayer oldPlayer;
    public VideoAdapter(OldPlayer oldPlayer) {
        //初始化 oldPlayer
        ②_____；
    }
    @Override
    public void playVideo() {
        System.out.println("适配器进行接口转换...");
        //调用 oldPlayer 对象的 play()方法播放视频
        ③_____；
    }
}

public class Test {
    public static void main(String[] args) {
        OldPlayer oldPlayer = new OldPlayer();
        //创建适配器实例，该实例封装了 OldPlayer 实例
        MediaPlayer player = ④_____；
        //调用适配器对象的 playVideo()方法播放视频
        ⑤_____；
    }
}
```

第 1 空:

第 2 空:

第 3 空:

第 4 空:

第 5 空:

题目 5 对象适配器模式

填空答案

① MediaPlayer

② `this.oldPlayer = oldPlayer`

③ `oldPlayer.play()`

④ `new VideoAdapter(oldPlayer)`

⑤ `player.playVideo()`

逐空解析

① 适配器 VideoAdapter 必须实现系统标准目标接口 MediaPlayer，对外提供统一播放规范。

② 构造方法接收老旧播放器对象，赋值给成员变量，对象适配器采用组合持有被适配类（不继承）。

③ 重写标准接口方法 playVideo()，内部调用老旧播放器不兼容的 play()，完成接口转换。

④ 客户端先创建老旧播放器，再传入适配器构造器生成适配后的标准播放器对象。

⑤ 客户端调用统一标准接口方法，底层自动适配老旧播放器。

模式核心小结

对象适配器：通过组合持有第三方旧类，实现目标接口做中转，不修改旧播放器、不改动业务代码，完成接口兼容。